Object Detection

Team Members:

1. Sanjith (2024027225)
2. Dhruv (2024027003)
3. Carlos Adison (2024016202)
4. Vamsi Krishna (2024026464)
5. Keerthika (2024029275)

Index

Abstract

Object detection is a key element of computer vision, with applications that span from surveillance to autonomous driving. This project aims to create a reliable human detection system using Python. By harnessing cutting-edge machine learning techniques and libraries, we strive to accurately identify and track individuals across various settings. Our solution emphasizes real-time processing and high precision, making it versatile for multiple applications. Designed with user accessibility in mind, we believe this system will significantly enhance automated monitoring and safety technologies.

Introduction

In recent years, object detection has gained prominence as a crucial component of computer vision, o ering solutions for numerous real-world problems. Human detection, a specialized branch of object detection, plays a pivotal role in applications like surveillance, crowd management, and personal safety. As technology advances, the demand for e cient, realtime human detection systems has intensified.

This project seeks to address these needs by developing a human detection system that is both accurate and accessible. Utilizing Python, one of the most popular programming languages for machine learning, we leverage powerful libraries such as OpenCV and TensorFlow to create a robust solution. Our focus is on achieving high detection accuracy while ensuring the system is lightweight enough for real-time applications.

The motivation behind this project stems from the increasing necessity for automated solutions that enhance safety and security in public and private spaces. By o ering an opensource model, we aim to foster collaboration and innovation within the computer vision community.

Our approach involves a comprehensive analysis of existing methodologies, selecting those best suited for human detection, and implementing them e ectively. By prioritizing modularity in our code structure, we ensure the system can be easily adapted for various use cases, further broadening its applicability.

Overall, this project not only aims to contribute to the field of computer vision but also aspires to provide a tool that can be integrated into diverse applications, ultimately improving the quality of automated monitoring systems.

SRS (System Requirement Specifications)

Functional Requirements

- **Hardware:**
  - A contemporary computer system equipped with at least 8GB of RAM GPU
  - support to facilitate accelerated processing **Software:**
- Python version 3.8 or above
  - Libraries: OpenCV, TensorFlow, NumPy, Matplotlib
  -

Non-Functional Requirements

- **Portability:**
  - The system should function seamlessly across various platforms, including Windows, macOS, and Linux.
- **Availability:**
  - Accessible to all under an open-source license.
- **Reusability:**
  - Code should be structured modularly to enable easy integration and modification for di erent applications.
- **Performance:**
  - Capable of real-time processing with minimal delay.
- **Accuracy:**
  - High detection accuracy in diverse lighting and environmental conditions.

Methodology

Code
from ultralytics import YOLO

import cv2 import cvzone import

math

cap = cv2.VideoCapture(0) #for webcam

cap.set(3, 640) cap.set(4, 480)

```
# cap = cv2.VideoCapture("Videos/motorbikes.mp4") #for video


model = YOLO("yolov8l.pt")


classNames = ["Human", "bicycle", "car", "motorbike", "aeroplane", "bus", "train", "truck", "boat",
              "traffic light", "fire hydrant", "stop sign", "parking meter", "bench", "bird", "cat",
              "dog", "horse", "sheep", "cow", "elephant", "bear", "zebra", "gira e", "backpack", "umbrella",
              "handbag", "tie", "suitcase", "frisbee", "skis", "snowboard", "sports ball", "kite", "baseball bat",
              "baseball glove", "skateboard", "surfboard", "tennis racket", "bottle", "wine glass", "cup",
              "fork", "knife", "spoon", "bowl", "banana", "apple", "sandwich", "orange", "broccoli",
              "carrot", "hot dog", "pizza", "donut", "cake", "chair", "sofa", "pottedplant", "bed",
              "diningtable", "toilet", "tvmonitor", "laptop", "mouse", "remote", "keyboard", "cell phone",
              "microwave", "oven", "toaster", "sink", "refrigerator", "book", "clock", "vase", "scissors",
              "teddy bear", "hair drier", "toothbrush"]
while True:
    success, img = cap.read()     results =
model(img, stream=True)
   for r in results:         boxes = r.boxes         for box in boxes:
#bounding box             x1, y1, x2, y2 = box.xyxy[0]
x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)


            w, h = x2-x1, y2-y1
cvzone.cornerRect(img, (x1, y1, w, h))


            #confidence             conf =
math.ceil((box.conf[0]*100))/100
```

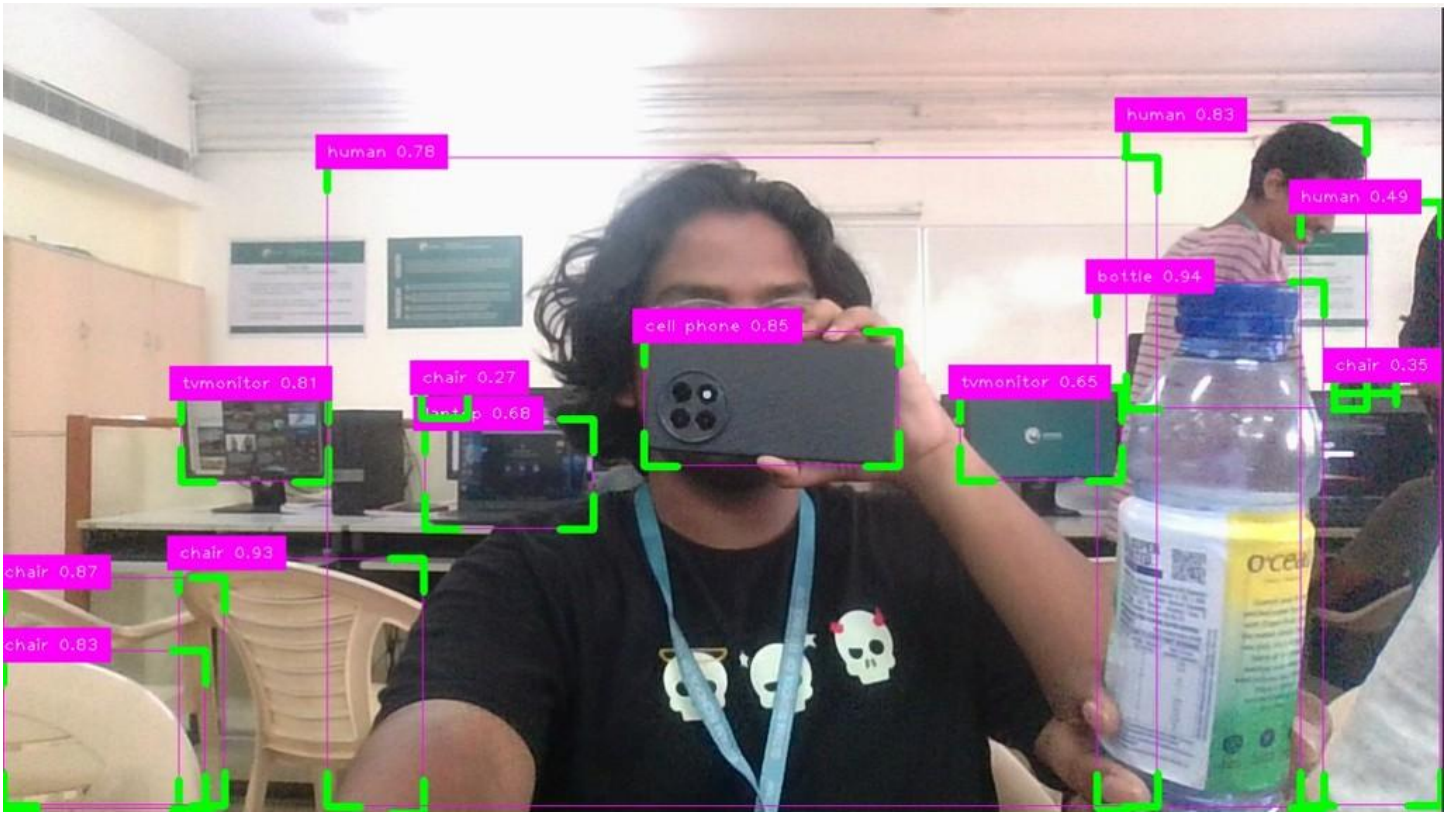
```
        print(conf)


        #classname
cls = box.cls[0]
        cvzone.putTextRect(img, f'{classNames[int(cls)]} {conf}', (max(0, x1), max(40, y1)), scale=1, thickness=1)


    cv2.imshow("Image", img)
    cv2.waitKey(1)
```

Results

Output



Conclusion and Future Scope

In summary, our project e ectively showcases the potential of human detection through Python. The system's impressive accuracy and real-time processing make it an ideal candidate for various uses, including surveillance and personal safety applications. Future improvements may entail expanding the dataset for enhanced accuracy

and integrating additional features such as crowd density estimation. Our open-source model promotes ongoing collaboration and innovation in the realm of computer vision.